
POLICY LOGIC FORGE

Turning regulation into logic you can check.

A technical report on the architecture of an evidence-carrying pipeline that converts policy documents into business rules, decision and process models, information schemas, and a proof-checked intermediate representation — deciding deterministically where it can, proving where a proof is possible, and refusing where the source does not support an answer.

AUTHOR

Reza Rahimi

REPOSITORY

`rrahimi-uci/policy-logic-forge`

DOCUMENT

Architecture & evaluation

SCOPE NOTE

This report describes what the system does and what it does not claim. It makes no assertion of legal correctness.

Summary

Most business systems do not implement a policy. They implement someone’s interpretation of a policy. A subject-matter expert identifies an obligation, an analyst rewrites it as a requirement, an architect turns it into a model, a developer writes the code, and an auditor later tries to reconstruct why the system behaved as it did. At every handoff a trigger, an exception, or a scope qualifier can quietly disappear, and nothing in the chain records that it has.

This report describes a pipeline built around a single ordering principle: *decide as much as possible with code, prove what can be proved, ask a model only what genuinely requires judgment, and send a human only what survives all three*. That ordering inverts the usual instinct of reaching for a model first and adding guardrails afterwards. Here the model is the last resort before a person, and every stage exists to make its job smaller.

Three properties distinguish the design. Every emitted artifact resolves back to the sentence it came from, and the verifying stage rebuilds that evidence from the raw corpus rather than trusting the citation a rule carries — so a hallucinated citation cannot pass. A frozen intermediate representation carries a formal semantics, and obligations such as decision-table disjointness are *proved* by exhaustive enumeration or returned as unknown, never assessed. And the system refuses: where the source does not support a construct, it declines to emit one and records a typed reason.

The final sections report what the evaluation actually found, including a pre-registered hypothesis that failed. A companion experiment tested whether the pipeline’s own refusals predict where a language model errs. They do not — and the derived selector performs worse than answering everything. That result is reported here in full, because a report that only carries its successes is not evidence of anything.

Contents

Summary	1
1 The translation gap	3
1.1 Four kinds of check	3
2 Pipeline architecture	4
2.1 The evidence spine	4
3 Orchestration	6
3.1 The readiness gate	6
3.2 What “independent” verification means	6
4 The compiler layer	7
4.1 Bounded proof, not a solver	8
4.2 Properties proved rather than tested	8
5 Generated artifacts	8
5.1 Module structure	9
5.2 RegDelta	10
6 Evaluation	10
6.1 Does it transfer off the domain it was built on?	10
6.2 A pre-registered hypothesis that failed	11
6.3 What the null result does not establish	12
7 Scope and limitations	12
7.1 Implementation scale	13

1 The translation gap

A policy is not an algorithm. It is prose written to be read by a person who will supply context, resolve ambiguity, and know which exceptions matter. Turning it into something a machine executes requires answering questions the prose never states explicitly:

- Who is the responsible actor, and who is merely mentioned?
- What event triggers the obligation, and when does it lapse?
- Which conditions are cumulative, and which are alternatives?
- What is the scope — which jurisdictions, parties, categories?
- Which sentence licenses each of the above?

Answering those is interpretation, and interpretation is where meaning gets lost. The failure mode that matters is not output that looks wrong. It is output that looks right: a well-formed decision table, correctly typed, elegantly rendered, resting on a condition nobody in the source document ever wrote.

The design constraint

A system that generates policy logic must be able to answer, for any element it produces, *which sentence licensed this* — and that answer must be checkable by something other than the component that produced it.

1.1 Four kinds of check

The architecture rests on an ordering of verification instruments by strength. Every question is pushed down to the strongest instrument that can settle it; only what no instrument can settle reaches a person.

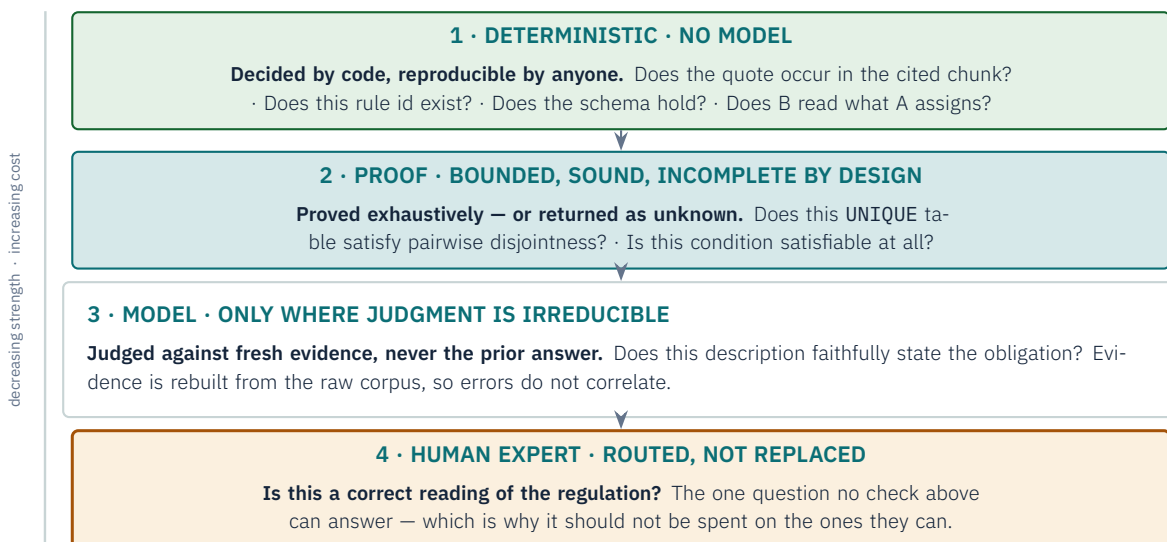


Figure 1. The verification ladder. Most questions a policy pipeline must answer are not matters of opinion — whether a quoted sentence occurs in the cited chunk is string resolution, whether a rule identifier exists is set membership, whether rule B reads what rule A assigns is a dataflow test. The difference this design makes is not a better judge; it is far fewer questions that need judging.

The economic argument follows from the ordering. Expert review is the genuine gold standard for legal correctness and cannot be replaced — but an expert cannot read several hundred rules by hand every time a policy changes. Spending that attention on questions a string comparison could have settled is the waste the ladder exists to eliminate.

2 Pipeline architecture

The system runs thirteen stages in order. The stage number and the agent identifier are always the same value, so `--stage 9` and `--agent agent_09` select the same stage. Each stage is a subprocess that checkpoints its output, so a run can resume from any boundary.

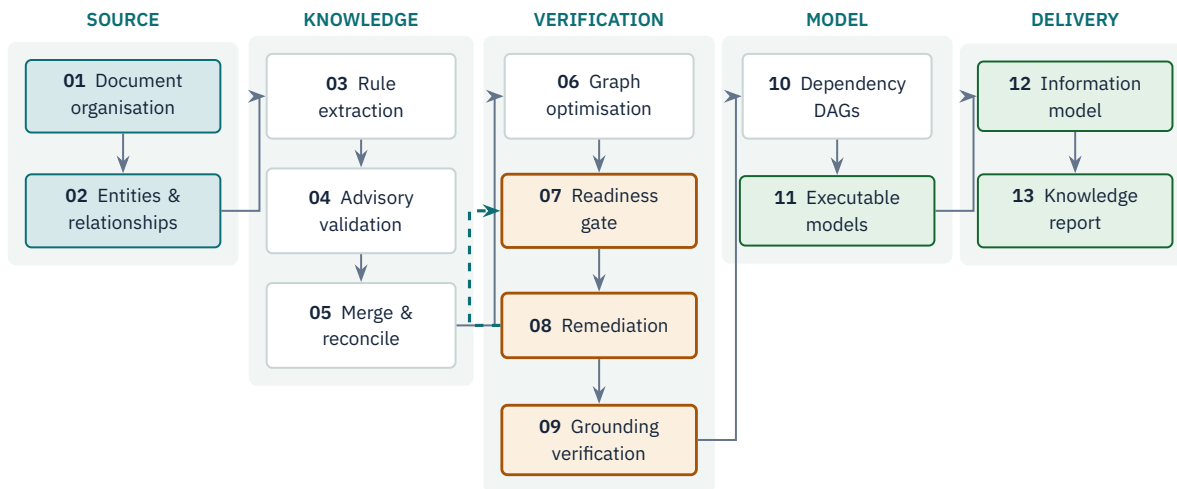


Figure 2. The thirteen stages, grouped by the responsibility each boundary carries. Amber stages form the verification block and share one directory because they read and write the same optimised graph. The dashed edge is the remediation loop: stage 08 patches only the fields that failed and re-enters the gate at stage 07, never re-extracting.

2.1 The evidence spine

Every artifact resolves back to the sentence it came from, and every step of that walk is checked rather than asserted. This is the single property the rest of the design exists to protect.

Traceability alone is cheap: any system can store a pointer alongside a generated rule, and a pointer that was never checked is exactly as convincing as one that was. What makes the spine load-bearing is the direction of the arrows. The forward path carries meaning from a passage into a rule, a symbol, an attribute, and finally into a decision table or a schema. The return path re-derives that evidence *independently*, from the raw corpus, using none of the intermediate structure the forward path built.

That asymmetry is what a hallucinated citation cannot survive. A rule asserting support from a sentence that does not exist will resolve forward perfectly well — the artifact renders, the types check, the table is well-formed — and fail only when something goes back to the document and looks.

Table 1. Stage responsibilities and primary outputs.

Stage	Responsibility	Primary output
01	Document organisation	agent_01-organized-documents/
02	Entity and relationship extraction	agent_02-entities/
03	Business-rule extraction	agent_03-rules/
04	Advisory rule validation	agent_04-validation/ (consumed by nothing; advisory by design)
05	Rules/entities merge	first complete knowledge graph
06	Graph optimisation	dedup + dependency analysis
07	Executable-readiness gate	readiness report; four invariants
08	Readiness remediation	focused field patches, only if 07 requests them
09	Grounding verification	claim-level certification
10	Dependency-DAG generation	dependency_dags.json, 100 % coverage
11	Executable model generation	DMN, BPMN, CMMN, SBVR + LExec compile
12	Business information model	LinkML schema, classes, typed attributes
13	Business knowledge report	self-contained HTML, zero network calls

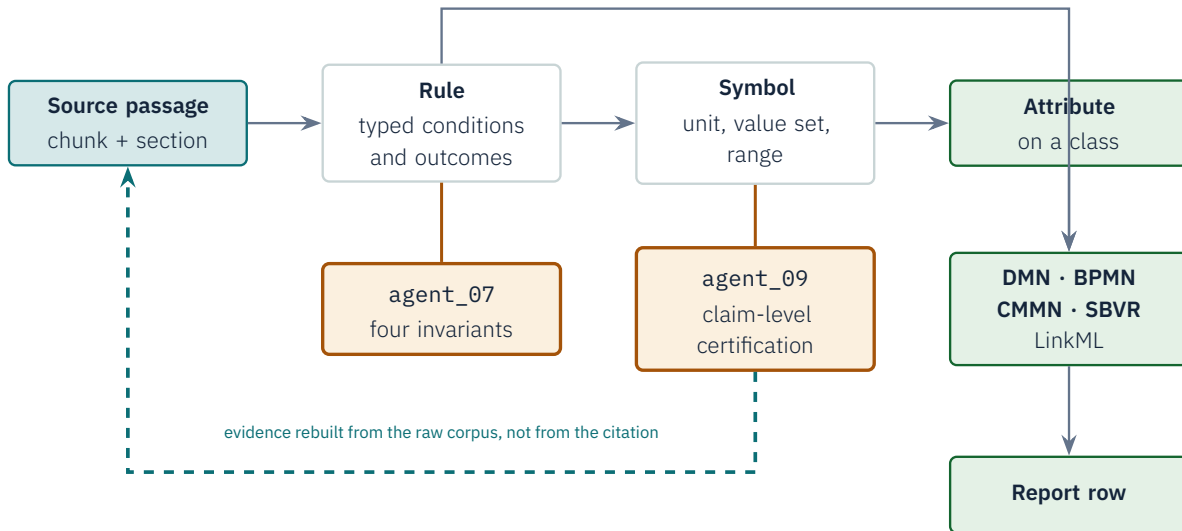


Figure 3. The evidence spine. Stage 03 extracts rules from source passages; stage 12 types symbols into class attributes. The dashed return path is the one that matters: stage 09 does not trust the citation a rule carries, it rebuilds the evidence from the raw corpus. On one real run that pass rejected 44 % of the dependency and conflict claims the pipeline’s own deterministic guard had already accepted.

3 Orchestration

A stage is a subprocess, so its exit code is the only signal the orchestrator can route on. There are three meanings, and one is deliberately overloaded.

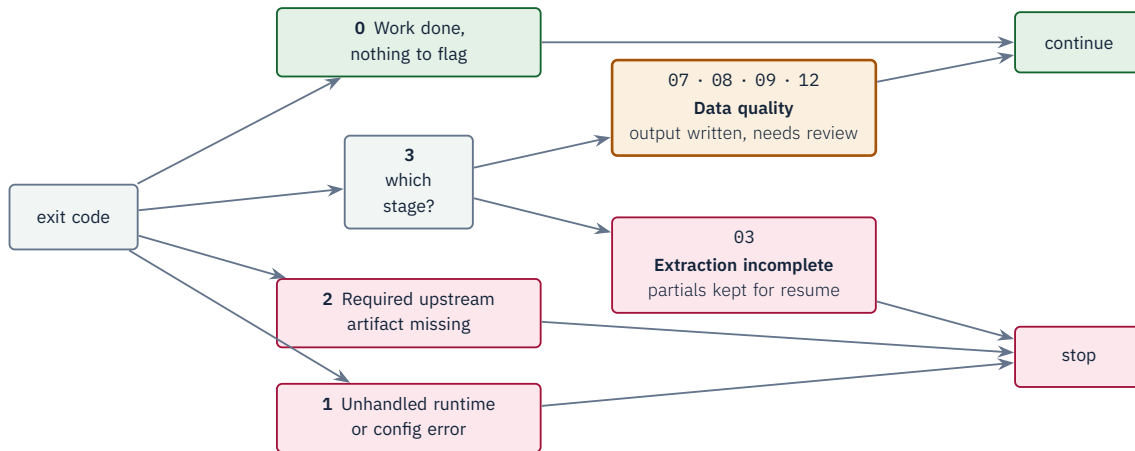


Figure 4. The exit-code contract. Code 3 branches on the stage that produced it: for the quality-gated stages it is a data-quality signal and the run continues carrying review flags, but for stage 03 it means extraction was incomplete and the run must stop so no later stage consumes a partial graph.

Two invariants follow, and both were violated in practice before they were enforced:

- **A stage may never exit 0 for work it did not do.** Stages 02 and 03 once printed “No documents found” and returned normally, so a run over an empty corpus was reported as successful through three further stages and only stopped at stage 05, blaming its own missing inputs.
- **A missing input is reported, not raised.** Stages 07, 08 and 09 died on an unhandled `FileNotFoundException`, which exits 1 — a code the orchestrator reads as a crash — and printed a traceback that said nothing about which stage to run first.

3.1 The readiness gate

Stage 07 completes every rule’s evidence-dependent fields through bounded evidence retrieval over the full corpus, then validates the graph against four deterministic invariants. Evidence retrieval builds an inverted token index — words, path tokens, and exception markers such as “except”, “unless”, “notwithstanding” — *before* any model call, so the search record can prove the whole corpus was inspected.

3.2 What “independent” verification means

Stage 09 certifies each source-derived field claim by claim against text it re-locates itself. Two design choices make it genuinely independent rather than a second opinion from the same source:

- Claims split into two *disjoint* paths. description, condition, outcome, party, scope and exception are model-verified against an evidence packet the stage rebuilds from the raw corpus. condition_logic and test_vector are excluded from model verification entirely — they are pipeline-derived values, not sentences any document states verbatim, and asking a model for a literal quote scored near-100 % false-insufficient in practice. Those are checked structurally instead.

Table 2. The four readiness invariants.

Invariant	What it checks
corpus_integrity	Every cited-section addition or removal between the baseline and final graph carries an explicit reason string.
naming_consistency	Zero entity-key violations against the canonical pattern $^{\wedge}[A-Z][A-Z0-9]*(?:_[A-Z0-9]+)*\$$.
schema_consistency	Zero <i>blocking</i> contract violations. Deliberately gated on blocking errors alone: folding in evidence and provenance gaps made this invariant fail on every run containing any review-required rule, pre-empting stage 08 before it could run.
referential_integrity	Zero dependency edges pointing at a <code>rule_id</code> that does not exist.

- Every verdict of *supported* or *contradicted* is downgraded to *insufficient evidence* unless the model’s cited quote resolves to a genuine substring of the real corpus text. A hallucinated citation cannot pass even when the model asserts support.

Coverage is not the same as outcome

`claim_coverage_percent` measures response-set *completeness* — did the verifier return exactly one verdict per claim, none missing, duplicated, or unexpected. It is a fail-closed integrity check on the verification pipeline itself, and is deliberately distinct from the supported/contradicted/insufficient counts. An incomplete verification run is a hard stop, never silently accepted.

4 The compiler layer

Alongside the review-oriented model outputs, the pipeline compiles the subset of rules it can represent into **LExec**, an intermediate representation with a frozen semantics. This path is additive and best-effort: a compile failure never blocks or replaces the review projection.

Table 3. LExec IR semantics, v1.

Property	Commitment
Null model	Kleene three-valued logic
Table boundary	unknown at a table boundary means <i>refuse</i> , not <i>default</i>
Exception reading	defeater-or
Operators	and or not is_null eq ne lt le gt ge contains in_binned_range
Effects	assignment only
Arithmetic	none — the IR has no arithmetic at all
Numeric theory	v2 number always lowers to real; an integer theory is never inferred from an integral-looking value

The absence of arithmetic is a deliberate scope boundary, and Section 6 reports what it costs.

4.1 Bounded proof, not a solver

Proof obligations — principally pairwise disjointness for UNIQUE hit-policy tables — are discharged by exhaustive enumeration over finite domains, capped at 10,000 assignments. The prover deliberately does not pretend to be a complete SMT backend.

Sound, and incomplete on purpose

The prover returns `unsat` only when the enumeration was *complete*. If the domain is unbounded or the cap is reached it returns `unknown` — never a green tick it cannot justify. Soundness is what is claimed; completeness is explicitly not.

4.2 Properties proved rather than tested

A test checks that one example behaved. The properties below are checked exhaustively over the bounded signature, so they hold for every input in it, not for the ones a test author happened to choose. All six are discharged by `proofs/check_properties.py` in a couple of seconds.

Table 4. Machine-checked properties. All hold.

#	Property	Method
1	$(T, \sqsubset)$ is a strict partial order — irreflexive, asymmetric, transitive	all 3,615 tuples
2	<code>reconcile_types(S)</code> returns the unique $\sqsubset$ -least element or $\perp$, never a coercion	all 1,940 subsets
3	<code>MONEY</code> $\parallel$ <code>PERCENTAGE</code> — neither refines the other	direct
4	Prover soundness: $\text{sat}(w) \Rightarrow w \models \varphi$ and $\text{unsat} \Rightarrow \varphi$ unsatisfiable	212 formulas $\times$ 48 assignments, 0 violations
5	Decidability on a bounded signature	0 unknowns, 0 disagreements with brute force
6	The dependency DAG set is a partition of the rule set	checked on a real 832-rule run

Property 3 is the legible one. `MONEY` and `PERCENTAGE` are both decimal numbers, and a system that quietly reconciles them will eventually read a 3 % rate as \$3. That cannot happen here, and *cannot* is the operative word: it is not a test that passed on the inputs someone thought of, it is a property that holds for every pair of types in the system.

Property 4 is the one that matters most for trust, because it is the checker checking itself.

5 Generated artifacts

Stage 11 emits standards-aligned models. Generation is gated on source support, not on whether a rule happens to have a dependency.

Table 5. Output formats and the question each answers.

Format	Question it answers	Note
DMN 1.3	What is decided, given these inputs?	Real emitter; hit policy carried through
BPMN 2.0	In what order, by whom?	Deliberately conservative — see below
CMMN 1.1	What may a knowledge worker do?	Intentionally simple
SBVR	What does the vocabulary mean?	Project-aligned profile, <i>not</i> full OMG interchange conformance
LinkML	What is the data shape?	Canonical form; JSON Schema and diagrams generated from it
LExec IR	What can be proved?	Proof-checked side channel, additive

The most useful thing the system does is refuse

BPMN generation requires an evidenced trigger, a responsible actor, and at least two explicitly ordered steps in the source. On privacy policies that test declines for the overwhelming majority of rules — which is the correct answer, not a shortfall, because a privacy policy states obligations, it does not describe workflows. Generating a diagram for every rule would have looked like far more progress and meant considerably less.

5.1 Module structure

Most of `utils/` is used directly by one or more stages with no interesting internal structure. The compiler layer is the exception: it is a genuine hub.

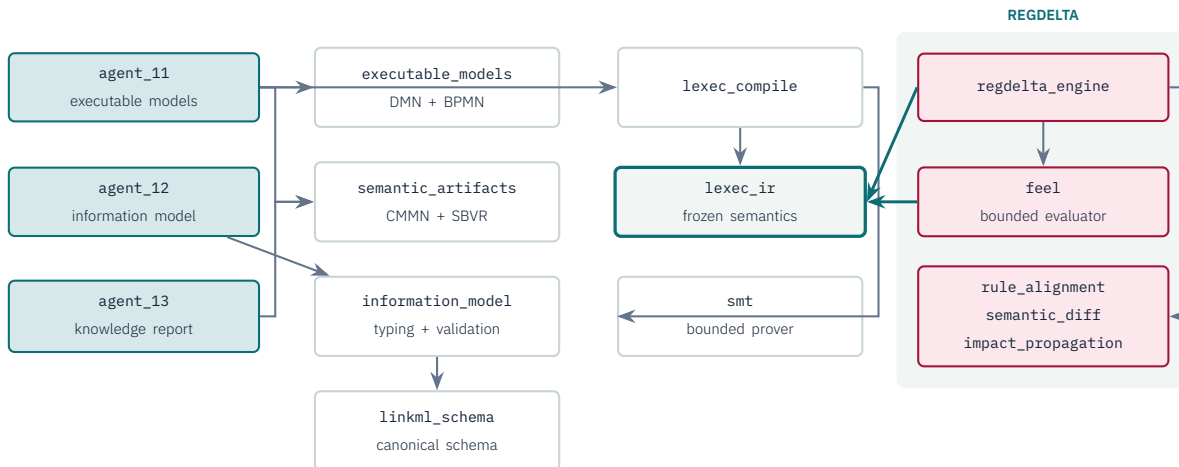


Figure 5. The compiler layer. Two independent emitter paths are easy to conflate: `executable_models` and `semantic_artifacts` produce the real DMN, BPMN, CMMN and SBVR outputs in every run and never touch the compiler, while `lexec_compile` produces a separate proof-checked IR for the representable subset. Teal edges mark `lexec_ir` as the shared hub.

5.2 RegDelta

Given two completed runs, RegDelta answers *what changed, for whom, and what else does it affect* — without re-running extraction.

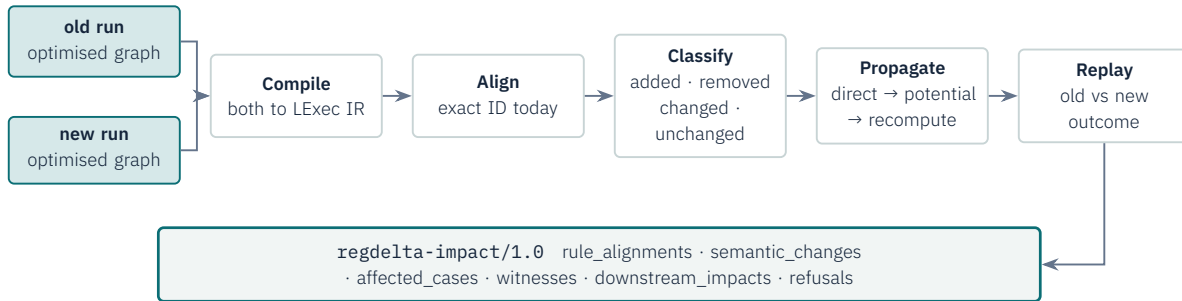


Figure 6. The differential-execution pipeline. Every rule in scope resolves to exactly one of *changed*, *unchanged*, *unresolved-review*, or *refused-unsupported-construct*; refused and review-held rules stay in the denominator of every reported metric, so nothing is silently dropped.

Two design choices are easy to get wrong by assumption:

- **Alignment is exact-identifier only today.** It implements the first rung of a five-rung ladder (exact ID → source-section signature → predicate structure → semantic similarity → explicit review). Two independently re-extracted runs of the same document will not align well yet; a renamed rule reads as a removal plus an addition.
- **Potential is narrative reachability, not proven dependency.** It walks stage 10’s DAG edges, which are extraction judgments about narrative dependency, not shared dataflow symbols.

6 Evaluation

6.1 Does it transfer off the domain it was built on?

The compile rate — how often an extracted rule lowers to the checkable IR — was only ever measured on privacy policies. A pilot measured the same number on an open benchmark of statutory reasoning, reporting a *raw* → *normalised* → *repaired* ladder with typed refusal codes so that a failure states its reason.

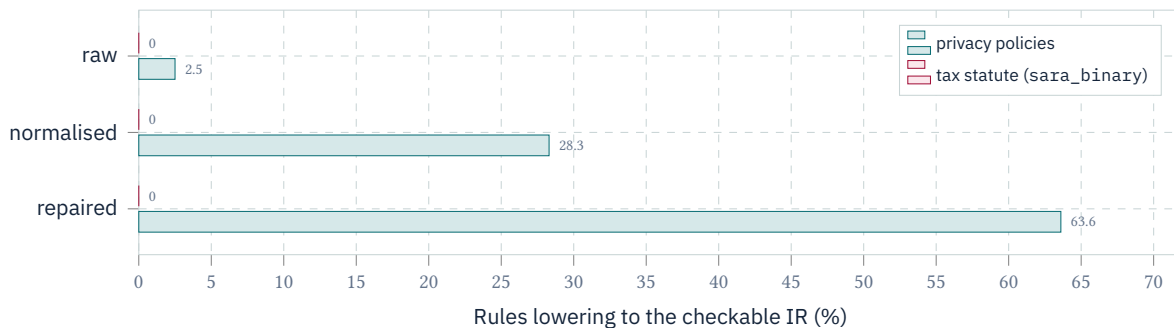


Figure 7. Compile-rate ladder. The statute compiles at 0 % at every rung, against 63.6 % for privacy policies. The headline is misleading, and the decomposition below is the actual finding.

The flat zero decomposes into two unrelated causes, one of them a defect in the system rather than a property of the task.

A hardcoded scope vocabulary

The IR’s scope dimensions are a five-name allowlist chosen for the two domains already run — `loan_types`, `transaction_types`, `occupancy_types` for mortgages and `user_categories`, `information_types` for privacy. A tax statute scopes rules by `case_types` and `configurations`, so lowering refuses them: not because the semantics cannot express the scope, but because the field name is not on the list. Adding those two names and changing nothing else takes the statute from 0 % to **33.3 %**.

This has a consequence worth stating plainly: **the 63.6 % privacy figure is partly an artifact of having tuned that allowlist to privacy**. Any cross-domain claim must fix this first.

The remaining refusals are entirely the absence of arithmetic. The split is total: of nine extracted rules, the three arithmetic-free ones (two eligibility, one exception) compiled — **3 of 3** — and the six arithmetic-bearing ones compiled **0 of 6**. Not one arithmetic-free statutory rule failed for a semantic reason.

6.2 A pre-registered hypothesis that failed

The refusal codes suggested something attractive: a deterministic, pre-execution difficulty signal obtained with no model call and no judge. If refused questions are the ones a model gets wrong, selective prediction becomes *sound by construction* rather than *calibrated by fitting*. Two hypotheses were fixed before running:

- H1** A case whose governing rule the IR refused is harder for a language model than one whose rule compiled.
- H2** The refusal signal selects abstentions at least as well as the model’s own confidence.

All 306 cases of the benchmark split were run twice — once at temperature 0 with self-reported confidence, once as a five-sample majority vote — with the model given the same statute the pipeline read, so the baseline is not a strawman.

Table 6. Accuracy by IR-coverage bucket, with Wilson 95 % intervals. Neither run separates the buckets, and the point estimate has the wrong sign in both.

Bucket	temp 0	5-sample	<i>n</i>	Meaning
covered	83.3 % [68.1, 92.1]	83.3 %	36	a compiled rule governs it
refused	90.0 % [83.0, 94.3]	88.2 %	110	a rule was extracted and refused
no_rule	87.5 % [81.5, 91.8]	90.0 %	160	extraction produced nothing

Both hypotheses fail

H1 is not supported. refused – covered is +6.7 pp (Fisher exact $p = 0.368$) and +4.8 pp ($p = 0.568$). Refused cases were, if anything, slightly *easier*.

H2 is refuted outright. Abstaining on refusals *raises* risk from 12.1 % to 16.7 %. The selector does not merely fail to help – it picks a worse-than-average subset.

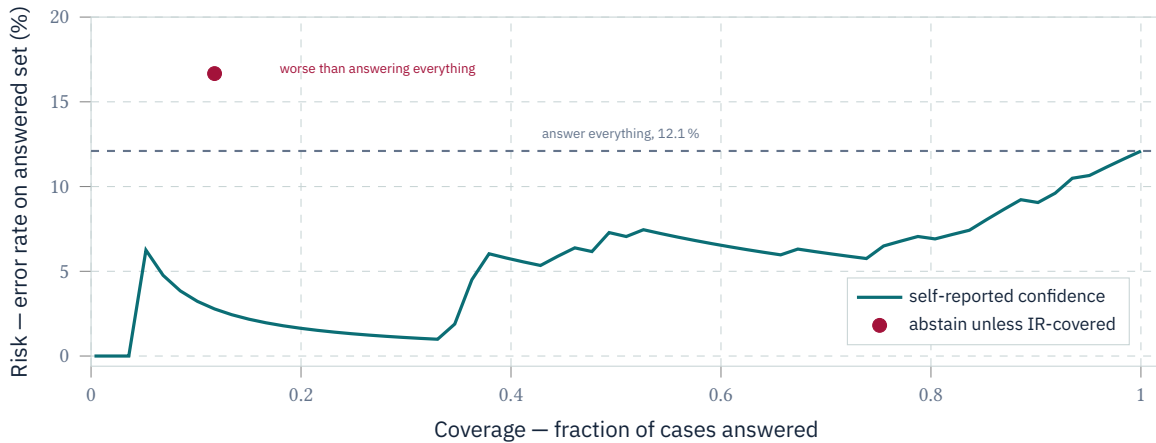


Figure 8. Risk–coverage. The refusal selector is a single operating point (red), and it sits *above* the answer-everything baseline: at 11.8 % coverage it carries 16.7 % risk where answering every case carries 12.1 %. The model’s own confidence, plotted as a full curve, reaches 2.8 % at the same coverage.

The mechanism shows up in a free ablation. A bare “does the claim assert a \$ amount” heuristic is flat – 87.4 % against 88.2 %. Arithmetic, which is precisely what the IR refuses, is not what the model finds difficult. **The IR refuses what is arithmetically routine and accepts what is semantically intricate**, which is close to the inverse of a language model’s difficulty profile.

Two findings emerged sideways. The model’s own confidence discriminates well here (AUC 0.774 and 0.801, $p < 0.0001$), which runs against the project’s own framing that model self-assessment cannot be trusted. And self-consistency was the *worse* selector, because the model stays consistent even when wrong.

6.3 What the null result does not establish

A null reported without its power is not evidence. Simulating the realised design, 80 % power is reached only against a gap of roughly 25 pp; a true 10 pp gap would have been detected 29 % of the time. So H1’s null means “no *large* effect”, not “no effect”. H2 does not depend on this – a selector that raises risk is a direction, not a significance question, and it replicated across two independent runs.

7 Scope and limitations

Credibility depends on drawing the line clearly.

The system does **not** establish a universal accuracy rate, complete concept recall, legal correctness, or automatic deployability. Those claims require curated expert-labelled benchmarks and target-

Table 7. Implemented, and explicitly not claimed.

Implemented and running	Not claimed
Staged CLI with checkpointed output at every boundary	It is a pipeline and a library, not a hosted governance platform
Structured rule contracts with field-level source references	Extraction quality still depends on source quality and model behaviour
Deterministic readiness invariants and targeted remediation	Readiness invariants check structure; they do not prove legal correctness
Independent claim-level grounding with literal quote resolution	Literal quote support is stricter than pointer presence, but is not legal entailment
Frozen IR semantics and a bounded proof layer	Bounded formal verification, not general theorem proving – sound but deliberately incomplete
DMN, selective BPMN, CMMN and SBVR-aligned outputs	SBVR is a project profile, not full OMG interchange conformance
Validated LinkML information model	Generated schemas still require environment-specific ownership decisions
Version-to-version change analysis with impact propagation	Alignment is exact-identifier only; a renamed rule reads as removal plus addition

environment validation, and none is made here.

The property that matters in regulated work

A deterministic check gives the same answer next quarter that it gave today. Re-run the pipeline on the same corpus and the same claims resolve the same way, for the same stated reasons. That is not true of a model verdict, and it is the property an audit actually depends on.

7.1 Implementation scale

Table 8. Source size. Test code slightly exceeds stage code.

Component	Files	Lines
agents/	14	15,392
utils/	32	10,790
cli/	3	1,152
tests/	64	15,654
Total		42,988

Every figure in this report is generated from committed result artifacts by `make_data.py`; no coordinate is transcribed

by hand. The proof table is reproduced by running `proofs/check_properties.py`, and the evaluation numbers by `measure_coverage.py` and `analyse.py` under `research/`.